

Sector skills project Report:

turbine_id	avg_temp	max_vibration
T-01	65.324224	11.9
T-02	65.156693	12.0
T-03	65.314775	12.0
T-04	90.581176	10.0
T-05	64.910816	12.0
T-06	64.804016	12.0
T-07	67.781065	25.0
T-08	64.857988	12.0
T-09	65.438462	12.0
T-10	64.408627	12.0
Turbines failing anomaly rules:		
['T-04', 'T-07']		

Findings

Turbine 4 and Turbine 7 are failing due to abnormal sensor readings, based on my script. [Evidence through screenshot of a run of my code above]

Architecture justification

I chose the message queue implementation of Apache Kafka / AWS Kinesis because all outputs like the alert system, dashboard, hot database and cold data lake, require the

anomaly detection process to be performed first ,so it buffers thousands of sensor readings between the IoT sensors (producers) and the streaming processors (consumers) allowing the streaming processors to run the anomaly detection algorithm on the sensor readings asynchronously [based on the number of streaming processors there are] . This increases scalability and reliability. The live dashboard fetches data from the hot database so that it's the most up-to-date. The cold data lake stores very old, rarely used/ rarely queried data. The alert system is decoupled from the databases because it gives an alert purely based on whether the current sensor reading is anomalous.

Cost Optimisation

The hierarchy of database storage is one way of optimising storage costs in this massive pipeline. Frequently accessed data is stored in the hot database, which provides faster response times but is more expensive. Less frequently accessed data is stored in the cold data lake, which has slower response times but is cheaper. This means the required size of the more expensive hot database is smaller, reducing costs and increasing efficiency.